

2.6 Assignment Operators

[This section corresponds to K&R Sec. 2.10]

The assignment operator `=` assigns a value to a variable. For example,

```
x = 1
```

sets `x` to 1, and

```
a = b
```

sets `a` to whatever `b`'s value is. The expression

```
i = i + 1
```

is, as we've mentioned elsewhere, the standard programming idiom for increasing a variable's value by 1: this expression takes `i`'s old value, adds 1 to it, and stores it back into `i`. (C provides several "shortcut" operators for modifying variables in this and similar ways, which we'll meet later.)

We've called the `=` sign the "assignment operator" and referred to "assignment expressions" because, in fact, `=` *is* an operator just like `+` or `-`. C does not have "assignment statements"; instead, an assignment like `a = b` is an expression and can be used wherever any expression can appear. Since it's an expression, the assignment `a = b` has a value, namely, the same value that's assigned to `a`. This value can then be used in a larger expression; for example, we might write

```
c = a = b
```

which is equivalent to

```
c = (a = b)
```

and assigns `b`'s value to both `a` and `c`. (The assignment operator, therefore, groups from right to left.) Later we'll see other circumstances in which it can be useful to use the value of an assignment expression.

It's usually a matter of style whether you initialize a variable with an initializer in its declaration or with an assignment expression near where you first use it. That is, there's no particular difference between

```
int a = 10;
```

and

```
int a;  
  
/* later... */  
  
a = 10;
```

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail feedback](#)